# Cómo iniciar un proyecto con un LLM — Paso a paso (v1.3.1)

**Versión:** `1.3.1`
**Fecha:** 2026-10-18
**Archivo:** `INICIAR-UN-PROYECTO-PASO-A-PASO-v1.3.1.md`
**Propósito:** Un procedimiento repetible, comprensible para principiantes, para iniciar un **proyecto de software pequeño o grande** que desarrollarás *junto con* un agente de programación, como Codex, OpenCode o Claude Code.

> **Qué hace esta guía:** Te lleva desde una idea hasta una carpeta de proyecto real, con control de versiones (el **andamiaje inicial**, o *scaffold*), instrucciones útiles, contratos de datos cuando correspondan, habilidades (*skills*) aprobadas y servidores MCP aprobados. **No establece una estructura de carpetas permanente.** El proyecto y su documentación evolucionarán mientras tú y tu agente lo desarrollan.

## Antes de comenzar

- **Tú** decides qué quieres conseguir y apruebas las decisiones importantes. **El LLM** ayuda a planificar y redactar archivos. **El agente de programación** trabaja en la carpeta real del proyecto y puede construir y probar el software.
- Un **chat con un LLM** ayuda a redactar planes incluso antes de crear la carpeta. Un **agente de programación** (Codex, OpenCode, Claude Code, etc.) normalmente trabaja con los archivos de esa carpeta.
- **Skill o habilidad:** instrucciones o procedimientos reutilizables para un agente. **Servidor MCP:** conexión que pone herramientas o datos a disposición del agente. No son equivalentes.
- **Scaffold o andamiaje:** directorio, archivos, configuración y pruebas iniciales; no es la aplicación terminada.
- Necesitas una terminal, Git y un agente de programación. Algunas habilidades requieren Node.js/npm (`npx`). Instala solamente las herramientas que necesite la tecnología seleccionada. Los comandos de ejemplo indican la terminal correspondiente. El agente de programación debe producir **tres** mecanismos de instalación: Bash, PowerShell y CMD, y verificar cada uno cuando sea posible.
- **Sigue los pasos en orden.** Cuando diga *si corresponde*, anota expresamente **No aplica** y continúa. Un proyecto pequeño no necesita obligatoriamente una habilidad, un servidor MCP, una base de datos, una interfaz gráfica o una biblioteca de validación.
- **Nunca incluyas contraseñas, tokens, credenciales, datos confidenciales de producción o datos personales en prompts, documentos o Git.** Utiliza ejemplos sintéticos o anonimizados.

---

## Elige la configuración más pequeña que sirva para tu proyecto

**No crees archivos solo para cumplir con una plantilla.** Los pasos se aplican a todos los proyectos, pero la documentación debe ser proporcional al trabajo. Si algo es opcional, escribe **No aplica** en tus notas de planificación y continúa.

| Tipo de proyecto | Empieza con | Añade cuando se necesite |
|---|---|---|
| Script pequeño o experimento | `README.md`, `AGENTS.md`, programa ejecutable y Git | `PROJECT_SPEC.md` breve; `ARCHITECTURE.md` si existen reglas importantes |
| Aplicación con interfaz de usuario | Lo anterior, más `DESIGN.md` | Pantallas, reglas de componentes y accesibilidad |

| Tipo de proyecto | Empieza con | Añade cuando se necesite |
|---|---|---|
| Proyecto de datos, importación, API o salida de LLM | Lo anterior, más `DATA_CONTRACTS.md`, `schemas/` u otros contratos pertinentes y pruebas de validación | Migraciones y verificaciones de compatibilidad |
| Aplicación compleja o colaborativa | Lo anterior, más `ARCHITECTURE.md`, habilidades e integraciones apropiadas | Más pruebas, CI, registros de decisiones y revisiones |

**Cómo usar esta guía:** Sigue los pasos numerados. *Tú* apruebas las decisiones; el *chat de planificación* redacta las especificaciones; tu *agente de programación* crea y modifica los archivos del repositorio real. Puedes adaptar cada prompt. Cada paso indica su resultado esperado. No necesitas preparar una montaña de documentos para un script pequeño.

## ¿Para qué sirve cada archivo del proyecto?

| Archivo o carpeta | Destinatario | Función | Cuándo crear o actualizar |
|---|---|---|---|
| `README.md` | Personas | Qué hace el proyecto y cómo ponerlo en marcha | Andamiaje inicial y cambios para usuarios |
| `PROJECT_SPEC.md` | Tú y el agente | Alcance, criterios de aceptación y objetivos | Planificación y cambios de requisitos |
| `AGENTS.md` | Agentes de programación | Comandos, convenciones, restricciones y referencias a otras reglas | Después de crear archivos reales y cuando cambien los procesos |
| `DESIGN.md` | Agente / participantes de UI | Valores concretos de diseño y reglas de interacción | Cuando existe o cambia una interfaz |
| `ARCHITECTURE.md` | Agente / responsables técnicos | Reglas invariantes y razones de diseño que no son evidentes | Cuando cambien decisiones arquitectónicas |
| `DATA_CONTRACTS.md` | Productores, consumidores y agente | Significado y compatibilidad de los datos intercambiados | Cuando cambien las interfaces de datos |
| `schemas/` | Programas y validadores | Contratos JSON legibles por máquinas, cuando corresponda | Cuando exista o cambie una interfaz JSON estructurada |
| `skills/` | Agente | Procedimientos reutilizables **locales del proyecto**, cada uno con `SKILL.md` | Cuando se necesite un procedimiento repetitivo particular |
| `SKILLS.md` | Tú y el agente | Inventario de habilidades seleccionadas, instaladas y globales | Cuando cambien habilidades o versiones |
| `MCP_SERVERS.md` | Tú y el agente | Inventario, permisos y conexión de MCP, **sin secretos** | Cuando cambien los servidores o su configuración |
| `scripts/scaffold.sh` | Tú y el agente | Configuración inicial reproducible, no estructura permanente | Creación del andamiaje o cambios deliberados de plantilla |

**Distinción importante:** `README.md` explica el proyecto a las personas; `AGENTS.md` indica cómo debe trabajar el agente *en este repositorio*; `SKILL.md` describe un procedimiento reutilizable en múltiples repositorios. No es necesario copiar una habilidad global a la carpeta `skills/` de cada proyecto; documenta su nombre, origen y forma de uso en `SKILLS.md`. Algunos agentes utilizan otros nombres de archivo o mecanismos de importación: comprueba su funcionamiento real, sin suponer que todos cargan `AGENTS.md` automáticamente.

# EMPIEZA AQUÍ — Un solo mensaje para toda la planificación

**Primero:** Abre una conversación nueva con ChatGPT, pega el siguiente mensaje y describe brevemente tu idea. **ChatGPT planifica contigo; Codex, OpenCode, Claude Code u otro agente programador crea los archivos reales después.** No necesitas ejecutar 18 mensajes por separado: esta guía también te permite revisar o reanudar cada paso.

> Actúa como facilitador interactivo de la planificación de mi proyecto. Soy principiante. Sigue esta norma v1.3.1. Haz una pregunta importante a la vez, conserva un registro de decisiones, cuestiona los supuestos mediante una entrevista obligatoria y explica las opciones técnicas más sencillas con sus ventajas y desventajas. Distingue mis requisitos, tus recomendaciones, los supuestos y lo que no has verificado. Prepara y revisa especificaciones, diseño de interfaz si aplica, arquitectura y contratos de datos si aplican. Evalúa primero mis habilidades y servidores MCP requeridos, después las recomendaciones opcionales; examina conflictos y seguridad. Durante la planificación no instales herramientas ni generes el proyecto real. Antes de entregar el trabajo al agente, prepara un informe de preparación, solicita mi aprobación y entrégame un paquete estructurado descargable para Codex, OpenCode o Claude Code. Solicita autorización ante decisiones importantes, riesgos de seguridad, sustituciones de herramientas y desviaciones de los requisitos; en otros casos, guíame por los pasos. Mi idea: [DESCRIBIR IDEA].

**Finalización de la planificación:** Apruebas la especificación definitiva y el paquete de transferencia. Entonces el agente programador ejecuta el plan en un repositorio local real.

# Reglas obligatorias aprobadas en la entrevista de requisitos

| Tema | Norma |
|---|---|
| Responsabilidades | ChatGPT planifica; un agente programador ejecuta el andamiaje y el desarrollo posterior. |
| Autonomía | El agente decide detalles técnicos rutinarios; solicita permiso ante cambios importantes, conflictos, riesgos de seguridad o modificaciones de decisiones aprobadas. |
| Entrevista | Es obligatoria la evaluación crítica de requisitos, proporcional a la complejidad. Se documentan bloqueos y asuntos sin resolver. |
| Capacidades básicas | Evalúa entrevistas críticas, búsqueda de habilidades, conversión de PRD a incidencias y revisión arquitectónica. Da prioridad a las herramientas elegidas por el usuario; consulta antes de sustituir. |
| Documentación | Son obligatorios `README.md` e instrucciones útiles para el agente (`AGENTS.md` o equivalente verificado). Otros documentos son condicionales; registra «No aplica». |
| Git | Commits, política de ramas, versiones y pruebas pertinentes. Pide permiso antes de operaciones destructivas o publicaciones. |
| Seguridad | Evalúa sensibilidad, transmisiones externas, permisos y secretos. Nunca publiques credenciales ni datos restringidos sin autorización. |
| Plataformas | Proporciona archivos de preparación para **Bash, PowerShell y CMD de Windows** en **todos** los proyectos; prueba lo posible y marca los demás como NOT VERIFIED. La equivalencia de los scripts no garantiza compatibilidad de la aplicación. |
| Evidencia | Informa PASS / FAIL / SKIPPED / NOT VERIFIED; nunca afirmes haber ejecutado una prueba no realizada. |
| Evolución | El árbol del proyecto puede cambiar. Revisa documentación, contratos y pruebas afectados. Los cambios arquitectónicos importantes requieren aprobación. |
| Norma viva | Las revisiones requieren aprobación y evidencia; conserva versiones, historial y ediciones sincronizadas en inglés y español. |

## Tres hitos de finalización distintos

No declares un proyecto *listo* sin indicar **qué hito** se alcanzó:

| Hito | Quién lo confirma | Evidencia necesaria |
|---|---|---|
| **Planificación completa** | Tú, con ChatGPT | `PROJECT_SPEC.md` aprobado, borradores pertinentes de diseño, arquitectura y contratos, riesgos, decisiones y paquete descargable de transferencia |
| **Andamiaje completo** | Agente de programación, con tu revisión | Repositorio creado, archivos reales, primera versión en Git, scripts de configuración y `SCAFFOLD_READINESS.md` con resultados |
| **Listo para desarrollar** | Tú y el agente | Instrucciones del agente funcionales, comprobaciones aplicables verificadas, bloqueos identificados y primera tarea definida |

**Un hito no sustituye a otro.** Un ZIP de planificación no es un proyecto construido; un commit correcto no demuestra que pasaron las pruebas.

## Reglas de aprobación (para todo el proceso)

**El agente puede avanzar sin volver a preguntar:** escoger detalles ordinarios dentro de los requisitos aprobados, añadir pruebas rutinarias, corregir problemas reversibles de configuración, actualizar documentación pertinente y realizar commits cuando ya cuente con autorización expresa.

**Debe solicitar tu aprobación:** cambiar requisitos fundamentales o arquitectura, sustituir una habilidad o MCP elegidos por ti, ampliar acceso a datos o transmisión externa, contratar servicios de pago, borrar datos, reescribir historial Git, publicar/desplegar o distribuir trabajo a subagentes. Registrar aprobaciones en `DECISIONS.md` o en el historial de decisiones del paquete.

**Si no hay aprobación:** detener únicamente la operación afectada; continuar con tareas independientes seguras y marcar el bloqueo. El silencio no equivale a autorización.

## FASE A — Decide qué vas a construir

### PASO 1 — Explica tu idea con palabras sencillas

Escribe de 2 a 5 oraciones. ¿Qué debe hacer? ¿Quién lo utilizará? ¿Cuál es el resultado más importante? No necesitas vocabulario técnico.

**Tu acción:** Escribe o dicta tu idea aproximada en un chat con un LLM.

**Resultado:** Una descripción breve e informal.

### PASO 2 — Convierte la idea en una especificación pequeña y comprobable

**Copia y pega en el LLM:**

> Esta es mi idea: [PEGA LA IDEA]. Ayúdame a crear la especificación de la primera versión. Describe el propósito, los usuarios previstos, entre 3 y 6 funciones esenciales, qué queda *fuera del alcance*, cómo comprobaremos que la primera versión funciona, una pila tecnológica recomendada y una estructura inicial sencilla de carpetas. Separa los hechos que proporcioné de tus suposiciones. Pregunta solamente por decisiones que alteren sustancialmente el plan. Guarda la especificación aprobada como `PROJECT_SPEC.md`.

Lee, corrige y aprueba el borrador. Empieza por la **versión útil más pequeña**; podrás añadir funciones después.

**Resultado:** `PROJECT_SPEC.md` aprobado.

## PASO 3 — Aclara requisitos inciertos (entrevista crítica)

Una habilidad de *grilling* formula preguntas enfocadas para descubrir supuestos ocultos. Puedes hacer la entrevista en el chat **sin instalar todavía ninguna habilidad**. Más adelante, `grill-with-docs` puede revisar y documentar las decisiones dentro del repositorio.

**Copia y pega:**

> Entrevístame sobre el proyecto propuesto. Concéntrate en requisitos ausentes, supuestos contradictorios, usuarios, entradas y salidas, casos extremos, criterios de éxito y restricciones. Haz una sola pregunta importante cada vez. Mantén un registro breve de decisiones y actualiza `PROJECT_SPEC.md` después de que apruebe las respuestas. Termina cuando el proyecto esté suficientemente claro para crear el andamiaje.

**Resultado:** Requisitos más claros y un registro breve de decisiones o preguntas pendientes. **No tienes que resolver todos los detalles futuros de implementación.**

## PASO 4 — Redacta los tres documentos de reglas del proyecto

Describen asuntos distintos. **Crea documentos cortos y útiles, no extensos textos especulativos.**

| Documento | Pregunta que responde | Cuándo se necesita |
|---|---|---|
| `DESIGN.md` | ¿Cómo deben verse y comportarse las pantallas? | **Si hay interfaz de usuario** (web, escritorio, móvil o interfaz CLI interactiva significativa) |
| `ARCHITECTURE.md` | ¿Cómo se organiza el sistema y qué reglas nunca deben incumplirse? | **Todos los proyectos**; muy breve para scripts pequeños |
| `DATA_CONTRACTS.md` | ¿Qué datos entran, salen o pasan entre componentes y qué significan? | **Proyectos con intercambio estructurado de datos, archivos, API, configuración o salida estructurada de LLM** |

**Copia y pega:**

> A partir de `PROJECT_SPEC.md`, redacta `ARCHITECTURE.md` (partes del sistema, invariantes importantes y decisiones tecnológicas). Si hay interfaz de usuario, redacta también `DESIGN.md` (valores de diseño, distribución, componentes, estados de carga y error, adaptación a pantallas y accesibilidad). Determina si hay entradas o salidas de datos estructurados; en ese caso, redacta `DATA_CONTRACTS.md` (productores, consumidores, formatos, significado y reglas de validación). Ajusta la extensión al tamaño del proyecto. Si algo no aplica, explica por qué. Señala contradicciones entre los documentos antes de pedirme aprobación.

En un script sencillo sin interfaz, `DESIGN.md` puede marcarse **No aplica** y omitirse. Un script diminuto puede registrar sus pocas reglas técnicas directamente en `AGENTS.md` y prescindir de un archivo `ARCHITECTURE.md` independiente; en sistemas no triviales, mantenlo explícito.

**Si creas `DESIGN.md`:** Solicita valores concretos, no adjetivos vagos: por ejemplo, metadatos iniciales YAML o una tabla con colores hexadecimales, familias tipográficas, escala de espaciado y radios de esquinas. Incluye distribución, componentes, accesibilidad por teclado, estados de carga/error y comportamiento responsivo. Así evitas que una sesión del agente cambie involuntariamente el diseño de otra. Para una UI pequeña, sé breve.

**Resultado:** Documentos de reglas adecuados y aprobados.

### PASO 5 — Define contratos de datos verificables por máquinas (cuando corresponda)

Si el proyecto recibe o produce datos estructurados, determina **su forma exacta antes de programar los módulos que los consumirán**. Ejemplos: importaciones CSV, solicitudes y respuestas API, configuración JSON, mensajes entre módulos y respuestas estructuradas de LLM.

Un **JSON Schema** describe restricciones sobre datos JSON: nombres, tipos, campos obligatorios, valores permitidos, límites numéricos y estructuras condicionales. **Por sí solo no demuestra todas las reglas del negocio** (por ejemplo, una fórmula actuarial, la unicidad entre registros o la igualdad de dos totales almacenados por separado). Utiliza código y pruebas adicionales para esas reglas. Para CSV, datos binarios, tablas de bases de datos y otros formatos, documenta también el contrato nativo correspondiente: JSON Schema no siempre es lo adecuado.

**Copia y pega:**

> Examina `PROJECT_SPEC.md`, `ARCHITECTURE.md` y `DATA_CONTRACTS.md` e inventaría cada productor, consumidor, entrada, salida y transformación de datos. Para interfaces JSON, crea esquemas en `schemas/`, indicando expresamente la versión de JSON Schema (prefiere Draft 2020-12 salvo incompatibilidad con las herramientas). Define campos obligatorios, tipos, enumeraciones, valores nulos, unidades, fechas, identificadores, rangos, campos opcionales y tratamiento de errores. Para formatos distintos de JSON, define un contrato equivalente. Genera ejemplos **sintéticos** válidos, inválidos y extremos e identifica reglas que requieren pruebas programadas adicionales. No inventes nombres de campos ni significados técnicos: enumera las dudas. Explica cómo validar y versionar los contratos.

**Estructura pequeña sugerida (solo lo necesario):**

```
schemas/
  inputs/
  outputs/
  config/
  examples/valid/
  examples/invalid/
tests/contracts/
```

Crea únicamente las subcarpetas utilizadas. Una API, por ejemplo, podría tener `schemas/api/create-order.schema.json`, y una configuración de agente `schemas/config/opencode.schema.json`. No fuerces JSON Schema sobre CSV ni sobre datos relacionales: utiliza contratos propios del formato. Define una política de compatibilidad: los cambios que rompan un productor o consumidor existente requieren revisión, migración o publicación coordinada. Determina si la versión del contrato figurará en el nombre, el identificador del esquema, la versión API u otro mecanismo explícito.

**Resultado:** `DATA_CONTRACTS.md` aprobado, esquemas pertinentes (por ejemplo, `schemas/inputs/import.schema.json`), ejemplos sintéticos y plan de validación. Si no hay intercambio estructurado, anota **No aplica**.

---

## FASE B — Selecciona habilidades del agente y conexiones MCP

**Distinción importante:** En esta fase **seleccionas y revisas** las herramientas. La instalación local se realizará **después de crear la carpeta y el repositorio Git**, para evitar problemas de orden.

### PASO 6 — Examina un conjunto básico de habilidades reutilizables

Estas son **candidatas para considerar en casi cualquier proyecto**, no una orden de instalarlas todas automáticamente. Antes de instalar, comprueba sus nombres y orígenes actuales: los repositorios pueden cambiar.

| Habilidad | Uso adecuado |
|---|---|
| `grill-me` **o** `grill-with-docs` | Cuestionar requisitos. Utiliza **una modalidad principal de entrevista**; `grill-with-docs` es especialmente útil cuando ya existen archivos y deben documentarse decisiones. |
| `find-skills` | Buscar habilidades cuando surja una necesidad. Descubrir una habilidad **no autoriza** instalarla automáticamente. |
| `to-prd` → `to-issues` | Convertir requisitos aprobados en un documento de requisitos de producto (PRD), y luego en tareas. Ejecuta `to-issues` **solo después** de aprobar el PRD y preparar un gestor de incidencias. |
| `improve-codebase-architecture` | Revisar o refactorizar código **ya existente**. **No** lo ejecutes obligatoriamente sobre un andamiaje vacío. |
| `setup-matt-pocock-skills` | Configurar el flujo auxiliar en un repositorio apropiado, ya inicializado, si utilizas ese conjunto. |

**Para consultar la oferta real (sin instalar a ciegas):**

```
npx skills@latest add mattpocock/skills --list
npx skills@latest add vercel-labs/skills --list
```

**Comandos de instalación específicos, como ejemplo** (ejecuta **después del Paso 12**, dentro del proyecto, solo si están aprobados y son compatibles con tu agente):

```
npx skills@latest add mattpocock/skills --skill grill-with-docs
npx skills@latest add vercel-labs/skills --skill find-skills
npx skills@latest add mattpocock/skills --skill to-prd --skill to-issues
npx skills@latest add mattpocock/skills --skill improve-codebase-architecture
npx skills@latest add mattpocock/skills --skill setup-matt-pocock-skills
```

**Alternativa opcional** a `grill-with-docs`:

```
npx skills@latest add mattpocock/skills --skill grill-me
```

El nombre `prd-to-issues` puede describir el proceso, pero las **dos habilidades indicadas** son `to-prd` y `to-issues`. Según el agente, las habilidades pueden ejecutarse con comandos de barra (como `/to-prd`), por nombre u otra interfaz. **Pregunta a tu agente cómo se invocan; no supongas que todos utilizan los mismos comandos.**

**Referencias:** https://skills.sh/mattpocock/skills · https://skills.sh/vercel-labs/skills/find-skills

## PASO 7 — Añade TUS habilidades obligatorias y solicita recomendaciones opcionales

**Copia y pega:**

> Las habilidades que exijo son: [NOMBRES, REPOSITORIOS O ENLACES]. Mantén mis elecciones separadas de las recomendaciones opcionales. Comprueba que sean compatibles con mi agente de programación. Recomienda únicamente habilidades adicionales que aporten funciones distintas. Explica cuándo usar cada una; todavía no instales nada.

**Dónde se guardan:** Una habilidad puede instalarse globalmente para el agente (compartida entre repositorios) o localmente en un proyecto. No copies habilidades globales a cada proyecto salvo que el agente lo exija o necesites fijar deliberadamente una versión local. Cada habilidad local normalmente incluye su `SKILL.md`; por ejemplo `skills/release-checklist/SKILL.md`. Mantén en `SKILLS.md` un inventario de origen, versión y ámbito de instalación.

**Resultado:** Una propuesta única de inventario: **exigida por el usuario**, **recomendada**, **aceptada/rechazada**, **motivo**, **origen de instalación** y **cuándo invocarla**. Será `SKILLS.md`.

### PASO 8 — Comprueba los conflictos entre habilidades antes de instalarlas

**Copia y pega:**

> Examina la documentación y configuración de cada habilidad seleccionada. Detecta duplicados, instrucciones contradictorias, funciones solapadas, incompatibilidades de dependencias o versiones, permisos y archivos de configuración en conflicto. Redacta `SKILL_CONFLICT_REPORT.md` con columnas: Habilidad A, Habilidad B, evidencia, gravedad (Crítica/Moderada/Leve), resolución recomendada y estado. Conserva mis habilidades obligatorias siempre que sea técnicamente posible y seguro; consúltame antes de sustituirlas. Distingue conflictos **verificados** de conflictos **posibles**. Si no encuentras ninguno, informa «No se detectaron conflictos entre habilidades» e indica lo que comprobaste. No instales habilidades con conflictos críticos pendientes.

**Resultado:** `SKILL_CONFLICT_REPORT.md` aceptado. Los solapamientos compatibles pueden conservarse si se delimitan sus responsabilidades.

### PASO 9 — Enumera TUS servidores MCP y considera recomendaciones opcionales

Un servidor MCP proporciona al agente funciones como operaciones con repositorios, bases de datos, servicios externos o documentación. **MCP es opcional:** un script local sencillo quizá no necesite ninguno.

**Copia y pega:**

> Mis servidores MCP obligatorios son: [LISTA, O «ninguno»]. Indica a qué se conecta cada uno, qué operaciones espero y si el agente seleccionado lo admite. Sugiere únicamente servidores complementarios e incluye sus fuentes oficiales y permisos necesarios. Todavía no los instales ni autorices.

**Resultado:** Plan `MCP_SERVERS.md` aprobado, o **No se necesitan servidores MCP**.

### PASO 10 — Revisa conflictos y permisos de los servidores MCP

**Copia y pega:**

> Revisa todos los servidores MCP seleccionados para detectar nombres de herramientas duplicados, escrituras simultáneas en conflicto, entornos de ejecución, transportes, versiones, puertos o autenticación incompatibles, accesos a archivos o redes y permisos excesivos. Redacta `MCP_CONFLICT_REPORT.md` con evidencia, gravedad, resolución y estado de verificación. No habilites conflictos críticos sin resolver. No solicites secretos en el chat ni guardes credenciales en archivos del repositorio.

**Resultado:** `MCP_CONFLICT_REPORT.md` aprobado o una entrada breve **No aplica** si no hay servidores. Revisar configuraciones no demuestra que la conexión funcione.

---

## Validación obligatoria del paquete de transferencia

ChatGPT debe proporcionar **un ZIP descargable** con archivos de planificación coherentes y útiles, no directorios vacíos. La selección depende de las necesidades del proyecto. Ejemplo:

```
project-handoff/
  START-HERE.md             # primeras instrucciones para el agente
  PROJECT_SPEC.md           # metas, alcance y criterios aprobados
  DECISIONS.md              # aprobaciones, supuestos y pendientes
  AGENT_HANDOFF.md          # instrucciones y acciones autorizadas
  ARCHITECTURE.md           # si corresponde
  DESIGN.md                 # si hay interfaz de usuario
  DATA_CONTRACTS.md         # si se intercambian datos estructurados
  schemas/                  # si corresponden contratos JSON
  SKILLS.md                 # habilidades solicitadas/aprobadas y fuentes
```

```
MCP_SERVERS.md           # servidores y permisos, sin credenciales
READINESS_PLAN.md        # comprobaciones posteriores al andamiaje
```

Antes de entregar, verifica: **(1)** especificación y arquitectura compatibles; **(2)** todos los requisitos obligatorios del usuario; **(3)** responsable y contrato de cada interfaz relevante; **(4)** conflictos críticos pendientes marcados como bloqueos; **(5)** estado solicitado, recomendado, aprobado, instalado o no verificado de habilidades/MCP; **(6)** ausencia de secretos; **(7)** permisos del agente y decisiones sujetas a aprobación; **(8)** primera acción localizable en `START-HERE.md`.

Este ZIP es un **resultado de planificación**, no prueba de que ya existen el andamiaje o las instalaciones. Si ChatGPT no puede crear el ZIP, debe entregar los documentos individualmente e informar de la limitación.

# FASE C — Aprobar la planificación y transferirla al agente programador

## PASO 11 — Comprobar la preparación antes de la transferencia

**Dile a ChatGPT:**

> Examina la especificación, `DESIGN.md` si aplica, `ARCHITECTURE.md`, `DATA_CONTRACTS.md` y esquemas pertinentes, habilidades y MCP seleccionados, restricciones de plataforma, seguridad y pruebas. Identifica contradicciones, decisiones pendientes y supuestos sin verificar. Clasifica cada hallazgo como BLOQUEANTE o NO BLOQUEANTE y señala PASS, FAIL, SKIPPED o NOT VERIFIED con evidencia. Continúa la entrevista hasta resolver los asuntos bloqueantes. Solicita mi aprobación explícita para transferir el proyecto.

**Resultado:** `PLANNING_READINESS.md` aprobado y registro de decisiones. No presentes los supuestos como hechos verificados.

## PASO 12 — Exportar el paquete estructurado de transferencia

**Dile a ChatGPT:**

> Genera un archivo descargable `PROJECT-HANDOFF-v0.1.0.zip` con el plan aprobado, `HANDOFF.md` para el agente, `PROJECT_SPEC.md`, `DECISIONS.md`, `PLANNING_READINESS.md`, `DESIGN.md` si aplica, `ARCHITECTURE.md`, `DATA_CONTRACTS.md` y `schemas/` si aplican, `SKILLS.md`, `MCP_SERVERS.md`, criterios de aceptación y `SECURITY.md` breve. Son especificaciones y configuraciones propuestas: no afirmes que el repositorio o las herramientas ya existen. No incluyas secretos. Adjunta un manifiesto de archivos incluidos y omisiones justificadas.

**Resultado:** ZIP descargable. Si no puedes crear un archivo, informa de la limitación; no inventes enlaces.

## PASO 13 — Abrir el agente programador y entregarle el paquete

Abre Codex, OpenCode, Claude Code u otro agente en el directorio previsto. Proporciónale el ZIP (o la carpeta extraída) y este mensaje:

> Lee `HANDOFF.md` y todos los documentos relacionados. Comprueba el sistema operativo real, programas instalados, permisos, restricciones de internet y herramientas disponibles. Presenta tu plan de ejecución. Respeta las decisiones aprobadas; pide autorización antes de sustituciones, desviaciones arquitectónicas, transmisiones restringidas o publicaciones. Crea el proyecto inicial y archivos funcionales, no solamente carpetas vacías. Indica lo que no puedas realizar.

**Resultado:** Plan ejecutable por el agente; el usuario autoriza cambios importantes.

### PASO 14 — Crear el proyecto real y registrar su primera versión

El agente crea el programa inicial, `README.md`, instrucciones `AGENTS.md` o equivalente, referencias a diseño, arquitectura y contratos, y un procedimiento de preparación repetible. **En todos los proyectos deben existir estas tres entradas:**

```
scripts/setup.sh          # Bash (Linux/macOS/WSL/Git Bash si aplica)
scripts/setup.ps1         # Windows PowerShell
scripts/setup.cmd         # Símbolo del sistema (CMD) de Windows
```

Los tres scripts deben ofrecer tareas de preparación equivalentes y documentadas; pueden compartir lógica interna. No supongas gestores de paquetes idénticos ni permisos de administrador. Protege los secretos con archivos locales ignorados por Git, verifica `.gitignore`, ejecuta `git init` cuando corresponda y crea un commit inicial. No sobrescribas trabajo, hagas `force push` ni publiques sin permiso.

**Resultado:** Repositorio real, tres scripts, archivos con contenido útil y primer commit identificable.

### PASO 15 — Instalar las capacidades aprobadas e inicializar el contexto

El agente instala **solo** habilidades y servidores MCP aprobados, en el ámbito global o local apropiado. Comprueba versiones, funciones y conexiones autorizadas; registra éxitos, errores y sustituciones aprobadas en `SKILLS.md` y `MCP_SERVERS.md`. Ejecuta `/init` **solo si el agente lo admite**; de lo contrario, utiliza su método documentado. Comprueba que las instrucciones enlacen `DESIGN.md`, `ARCHITECTURE.md`, `DATA_CONTRACTS.md` y habilidades aplicables. Las credenciales nunca se incluyen en Git.

**Resultado:** Contexto y herramientas preparados, o limitaciones marcadas explícitamente.

### PASO 16 — Verificar el proyecto inicial y explicar el siguiente paso

**Dile al agente:**

> Ejecuta las comprobaciones posibles: programa inicial, dependencias, configuración, ejemplos de esquemas, pruebas, invocación de habilidades, conexiones MCP autorizadas y scripts de preparación de las plataformas disponibles. Para cada comprobación indica PASS, FAIL, SKIPPED o NOT VERIFIED y muestra el comando o evidencia. No presupongas que funcionan plataformas que no has probado. Crea `SCAFFOLD_READINESS.md`, identifica el commit Git y proporciona instrucciones sencillas para ejecutar, probar y continuar el desarrollo. Registra los cambios verificados sin publicarlos salvo autorización.

Un FAIL debe corregirse o declararse. **NOT VERIFIED no equivale a PASS.**

**Resultado:** Base verificada, informe con evidencia, tareas pendientes y siguiente acción exacta para el principiante.

---

## FASE D — Desarrolla y conserva la salud del proyecto

### PASO 17 — Planifica la primera tarea pequeña de desarrollo

El andamiaje **no** es una aplicación completa. Solicita que el agente convierta la especificación aprobada en cambios pequeños con pruebas. Si escogiste el flujo de Matt Pocock, usa `to-prd` para preparar el PRD y `to-issues` **después** de aprobarlo y configurar el gestor de incidencias. Para un script pequeño, basta una lista sencilla de tareas; no hace falta un gestor formal.

**Copia y pega:**

> Lee las reglas del repositorio. Propón la función útil más pequeña que debemos implementar, con criterios de aceptación y una forma de probarla. Identifica qué archivos y contratos de datos afecta. Implementa un

> cambio revisable cada vez, ejecuta las pruebas pertinentes, resume los cambios y crea un commit solo después de que apruebe el resultado o te autorice explícitamente a hacerlo.

**Resultado:** Un ciclo incremental de desarrollo, no una generación gigantesca imposible de verificar.

## PASO 17A — Paralelizar opcionalmente incidencias mediante subagentes

Tras aprobar el PRD, utiliza `to-prd` y después `to-issues` (o equivalentes aprobados) para obtener incidencias con criterios de aceptación, dependencias, responsables e interfaces afectadas. **No asignes automáticamente cada incidencia a un agente diferente.**

**Dile al agente programador:**

> Examina las incidencias aprobadas de `to-issues`. Construye un grafo de dependencias e identifica tareas independientes que no modifiquen los mismos archivos y cuyas interfaces estén definidas. Propón un plan paralelo con un agente coordinador, una rama o worktree de Git por colaborador, responsables, orden de integración, pruebas y alternativa secuencial. Explica las posibles colisiones y el costo de integración. Solicita mi permiso antes de lanzar subagentes o activar servicios externos con costos. No ejecutes tareas dependientes simultáneamente sin un contrato de interfaz acordado.

**Reglas:** El coordinador controla el grafo de incidencias y la integración final. Los subagentes no modifican simultáneamente archivos compartidos sin coordinación. Las tareas independientes pueden ejecutarse en paralelo; las dependientes siguen su orden. Cada agente entrega cambios concretos, pruebas y resumen. El coordinador revisa la integración, repite las pruebas de integración y contratos, y documenta resultados. Si la herramienta no admite subagentes, trabaja en secuencia. Marca **No aplica** cuando el costo de coordinación supere el beneficio.

**Resultado:** Plan paralelo aprobado e incidencias integradas y verificadas, o alternativa secuencial documentada.

## PASO 18 — Permite que evolucione el árbol del proyecto, pero mantén sus contratos e instrucciones

Durante el desarrollo, el agente puede añadir carpetas, cambiar nombres de módulos, modificar dependencias o proponer otra arquitectura. **Es normal.** `scripts/scaffold.sh` documenta la **configuración inicial**, no obliga a restaurar continuamente el árbol original. Mantén `README.md`, `ARCHITECTURE.md`, `DESIGN.md` y `DATA_CONTRACTS.md` acordes con el software real.

**Antes de aceptar un cambio, pregunta:**

> ¿Este cambio afecta la arquitectura, la interfaz de usuario, los productores o consumidores de datos, los esquemas, las habilidades, los servidores MCP, los permisos o los comandos de compilación? Si es así, actualiza los documentos y pruebas correspondientes dentro del mismo cambio. Comprueba compatibilidad y migraciones antes de modificar interfaces de datos. Repite la revisión de conflictos si cambian habilidades o servidores MCP. Muéstrame los resultados de verificación y riesgos pendientes.

Repite la inicialización del agente o actualiza `AGENTS.md` cuando cambie significativamente la estructura; **revisa los resultados porque una inicialización automática puede sobrescribir instrucciones personalizadas**. Utiliza `improve-codebase-architecture` solamente cuando exista código sustancial y quieras evaluar su arquitectura. Registra puntos de recuperación en Git y conserva el historial.

**Resultado:** Un proyecto en crecimiento que tú y las futuras sesiones del agente puedan comprender.

---

## Compatibilidad de agentes: comprobar, no suponer

| Capacidad | Codex | OpenCode | Claude Code | Comprobación necesaria |
|---|---|---|---|---|
| Instrucciones del repositorio | Frecuentemente `AGENTS.md` | Depende de versión y configuración | Frecuentemente `CLAUDE.md` | Verificar que carga/importa las instrucciones |
| Orden de inicialización | Depende de la interfaz/versión | `/init` cuando exista | `/init` cuando exista | Consultar la ayuda de la versión instalada |
| Habilidades e invocación | Admitidas en entornos compatibles | Depende de versión/ complementos | Admitidas en entornos compatibles | Confirmar ruta, descubrimiento e invocación |
| Servidores MCP | En configuraciones compatibles | En configuraciones compatibles | En configuraciones compatibles | Probar permisos, transporte y una operación inocua |
| Subagentes paralelos | Depende de configuración | Depende de configuración | Depende de configuración | Verificar disponibilidad y costo; usar ejecución secuencial si no existe |

**Esta tabla orienta, pero no garantiza compatibilidad.** El agente debe verificar la versión instalada y documentar resultados observados. Las habilidades globales y locales tienen alcances distintos.

## Si algo falla: procedimiento de recuperación para principiantes

1. **No repitas indefinidamente el comando fallido.** Copia el error exacto y el comando al chat del agente, sin secretos.
2. Pregunta: *«Explícame el error de forma sencilla. ¿Falta un requisito, usé otra terminal, no tengo permisos, hay versiones incompatibles, fallan las credenciales o la red, hay conflictos de dependencias o existe un defecto en el código?»*
3. Pide la **corrección reversible más pequeña**, sus efectos y la manera de comprobarla; solicita aprobación previa si afecta seguridad, decisiones o historial publicado.
4. Repite la comprobación y registra el resultado real en `SCAFFOLD_READINESS.md`: **PASS / FAIL / SKIPPED / NOT VERIFIED**, indicando comando, plataforma y evidencia.
5. Si el entorno no permite la prueba, marca **NOT VERIFIED**, documenta una alternativa y continúa solo cuando no haya riesgos o dependencias bloqueantes.
6. Si la instalación queda dañada, consulta el último commit válido **sin perder cambios locales**. Revisa `git status` y solicita aprobación antes de operaciones destructivas.

**Casos particulares:** si no se descubre una habilidad, confirma nombre, ubicación y recarga el agente; si MCP falla, revisa permisos y credenciales sin revelarlas; si falla Git push, conserva commits locales; si no descargan PDF/ZIP, usa los Markdown guardados en GitHub o su opción de descarga.

## Prueba integral pequeña de esta norma

Antes de considerar verificada la guía en un agente específico, síguela una vez con un **proyecto de prueba sin datos sensibles** (por ejemplo, una utilidad que valide un JSON sintético). Conserva evidencia de que funcionaron la entrevista, transferencia, scripts, Git, inicialización, ejecución, pruebas de contrato si corresponden y reporte final.

**Lista para la prueba:**

- [ ] ChatGPT generó el paquete aprobado; el agente encontró `START-HERE.md`.
- [ ] El agente creó archivos reales y realizó el commit inicial.
- [ ] Existen Bash, PowerShell y CMD equivalentes; las plataformas no probadas figuran como NOT VERIFIED.

- [ ] Se registraron los resultados de ejecución, descubrimiento de habilidades, MCP y pruebas aplicables.
- [ ] El principiante sabe cuál es la siguiente acción y puede recuperarse de un error reversible provocado deliberadamente.
- [ ] Las ediciones inglesa y española contienen pasos y versiones equivalentes.

**Regla de evidencia:** La lista describe una prueba **pendiente de ejecución**, no afirma que se haya completado. Cuando alguien la ejecute, debe registrar fecha, entorno, comandos y resultados.

## Apéndice — prompts para mantener el contexto del agente

Al crear o actualizar `AGENTS.md`, una instrucción breve puede ser:

> Lee `PROJECT_SPEC.md` para conocer el alcance; `ARCHITECTURE.md` para las invariantes; `DESIGN.md` para normas de UI (si existe); `DATA_CONTRACTS.md` y `schemas/` para interfaces de datos (si existen); y `SKILLS.md` / `MCP_SERVERS.md` para las herramientas aprobadas. Consulta `scripts/scaffold.sh` solo para investigar o modificar la configuración inicial. Utiliza los comandos de compilación y prueba reales de este repositorio; nunca inventes archivos ausentes ni guardes secretos en Git.

Una habilidad local sencilla puede residir en `skills/<skill-name>/SKILL.md` e incluir nombre, descripción, cuándo usarla, procedimiento ordenado y comprobaciones. El descubrimiento automático de esa carpeta **depende de la herramienta**: confirma la ruta exigida en su documentación. Instala fuentes externas solo después de revisar su contenido y permisos.

Cuando cambies el árbol, **actualiza sus referencias**, pero no sobrescribas rutinariamente documentos mantenidos a mano al repetir la inicialización del agente. Revisa las diferencias y registra los cambios en Git.

---

## Lista rápida de comprobación

- [ ] Los objetivos y límites de la primera versión están en `PROJECT_SPEC.md`.
- [ ] `ARCHITECTURE.md` contiene reglas reales; `DESIGN.md` existe si hace falta una UI.
- [ ] Identifiqué productores y consumidores de datos; hay esquemas y pruebas sintéticas cuando corresponde.
- [ ] Distinguí mis habilidades y servidores MCP obligatorios de las recomendaciones opcionales.
- [ ] Revisé conflictos de habilidades y MCP; resolví los críticos antes de instalar.
- [ ] El andamiaje crea archivos reales y un proyecto inicial ejecutable.
- [ ] Git conserva al menos un commit inicial; no se han registrado secretos.
- [ ] El agente seleccionado puede leer las reglas y usar las herramientas aprobadas.
- [ ] Las pruebas iniciales pasan o están documentados los fallos y bloqueos.
- [ ] Comprendo que el árbol inicial es un punto de partida, no una estructura permanente.

## Todo el proceso en una línea

**Idea → especificación → aclarar requisitos → diseño/arquitectura/contratos de datos → seleccionar y revisar habilidades/MCP → andamiaje → primera versión en Git → configurar herramientas → inicializar el agente → verificar → desarrollar funciones pequeñas → actualizar documentos, esquemas, pruebas y Git conforme evoluciona el proyecto.**

---

## Control de versiones de ESTA norma reutilizable

Esta guía es una **norma viva independiente**, no el número de versión de los proyectos creados con ella.

- Utiliza `MAJOR.MINOR.PATCH`: **MAJOR** para cambios importantes incompatibles en el proceso; **MINOR** para nuevas fases o instrucciones compatibles; **PATCH** para correcciones editoriales.
- Cada publicación modifica la versión del **título**, **nombre de archivo** y **metadatos**, y añade una entrada al historial. Conserva las versiones anteriores en Git o en un archivo versionado. No sobrescribas una versión antigua con contenido nuevo.
- Commit recomendado: `docs(standard): release v1.3.1`; etiqueta opcional: `project-standard-v1.3.1`.
- Puedes utilizar esta norma con varios sistemas operativos, agentes y lenguajes de programación. Las instrucciones específicas de los agentes pueden cambiar; verifica su documentación oficial antes de ejecutarlas.

### Historial de revisiones

| Versión | Fecha | Cambios |
|---|---|---|
| **1.3.1** | 2026-10-18 | Estabilización: hitos, autorizaciones, validación de transferencia, compatibilidad de agentes, recuperación de errores, protocolo de prueba integral, scripts multiplataforma e historial corregido. |
| **1.3.0** | 2026-10-18 | Decisiones de 18 preguntas, mensaje maestro, transferencia estructurada, scripts para tres terminales, verificación con evidencia y subagentes paralelos opcionales. |
| **1.2.0** | 2026-10-18 | Se fusionaron conceptos de Project Bootstrap Kit en una guía única para principiantes: matriz de responsabilidades, habilidades globales frente a locales, valores concretos de diseño, configuraciones mínimas, portabilidad entre agentes, precauciones para repetir el andamiaje y separación entre documentación para personas y agentes. |
| **1.1.0** | 2026-10-18 | Se reorganizó como proceso literal para principiantes; se añadieron `PROJECT_SPEC.md`, condiciones claras para `DESIGN.md`, `DATA_CONTRACTS.md`, diseño y validación de JSON Schema, secuencia de instalación, comprobación del proyecto real, inicialización del agente y mantenimiento durante la evolución del proyecto. |
| **1.0.0** | 2026-10-18 | Primera versión formal: habilidades reutilizables, selección e instalación de MCP, revisiones de conflictos, andamiaje, contexto del agente y control de versiones con Git. |